

HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

Distributed Computing

Michael Kosyakov
Associate Professor

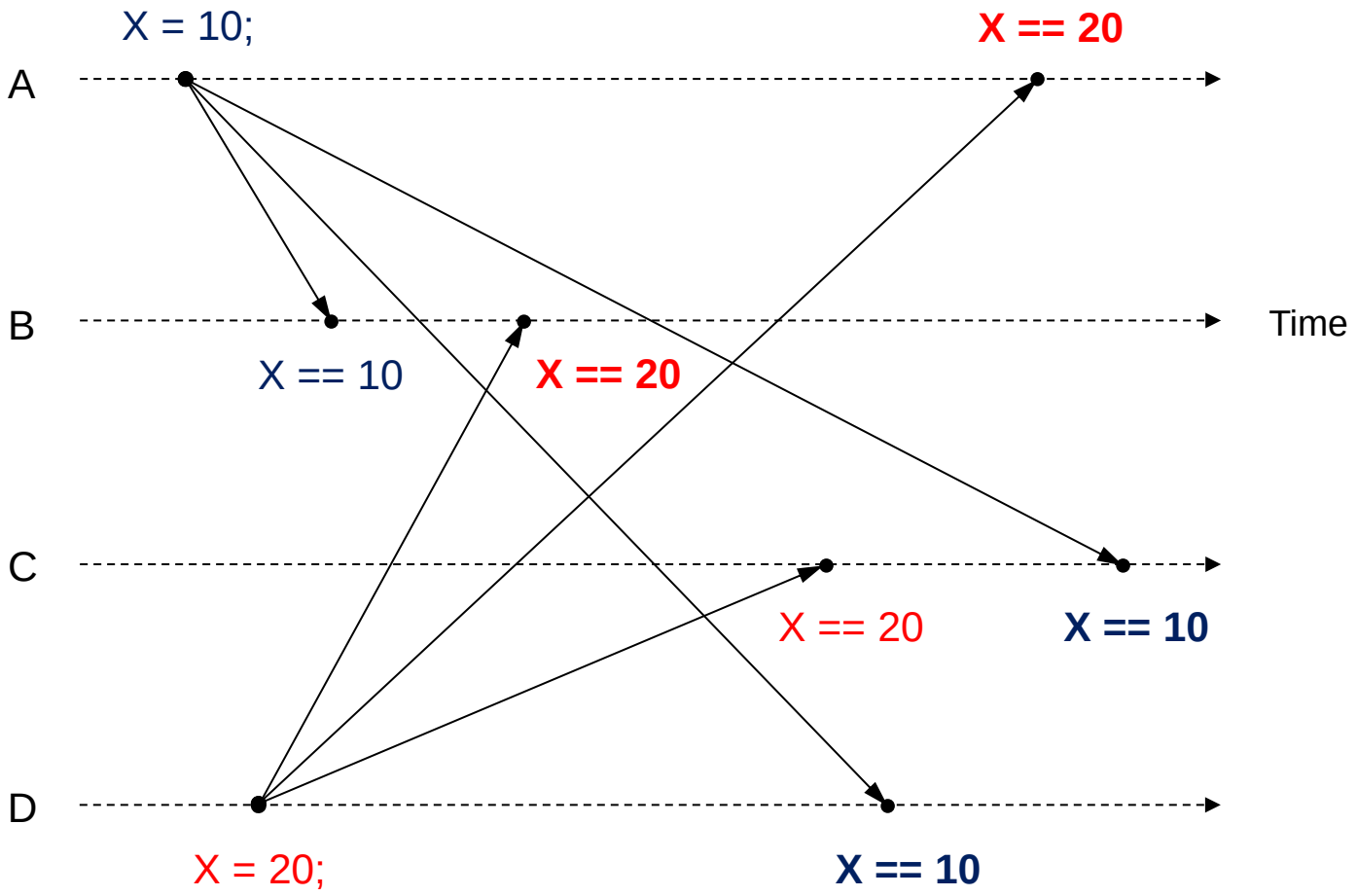
Denis Tarakanov
Assistant Lecturer

ifmo.distributedclass.bot@gmail.com



Distributed mutual exclusion algorithms

Mutual Exclusion vs Cache coherence



Essentially, the goal is the same as for cache coherence protocols, that is to maintain at-most-one writable copy of data, i.e. data updates should be seen in the same order by all processes (LWW using Lamport clock leads to lost updates)

Basic concepts

- **Critical section (CS)** is a part of a program with access to shared resources (data structures or devices), that should be used exclusively
- **Mutual exclusion** is a requirement that while a process is executing a critical section, no other process is executing the same critical section (having the same identifier)
- Lack of shared memory in distributed systems prevents the use of shared variables (e.g. semaphores)
  - Message passing is the sole means for implementing distributed mutual exclusion
- DME algorithms have to deal with unpredictable message delays and incomplete knowledge of the system state

Assumptions

- All processes run and communicate asynchronously
- Processes never fail
- Channels reliably deliver all messages and the network does not get partitioned
- Requirements for message ordering provided by communication channels (i.e. if channels are FIFO or not) are specific for each algorithm
- Accesses to shared resources come from a single critical section
  - It is not difficult to extend the algorithms presented below to work with more than one critical section

Source code with critical section



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

P_1

```
void P1(int R)
{
  while(true) {
  /* preceding code */;
  request_cs(R);
  /* critical section */;
  release_cs(R);
  /* subsequent code */;
  }
}
```

...

P_N

```
void PN(int R)
{
  while(true) {
  /* preceding code */;
  request_cs(R);
  /* critical section */;
  release_cs(R);
  /* subsequent code */;
  }
}
```

In the code of each process, the following parts can be distinguished

1. A critical section identified by R that has accesses to shared resources
2. Preceding and subsequent parts of the code, where there are no such accesses

Process states

- A process can be in one of the following three states:
- (1) Requesting the CS. The process is blocked until it gets permission to enter CS and cannot make further requests for the CS
- (2) Executing the CS. No process executes the CS forever
- (3) Executing outside the CS. The process can execute outside the CS for as long as desired
- The only possible state transitions are: from 3 to 1 (by invoking **`request_cs`**), from 1 to 2 (by returning from **`request_cs`**) and from 2 to 3 (by invoking **`release_cs`** and returning from the function call)

Requirements for DME algorithms



- (1) Safety. At any instant only one process can execute the CS
- (2) Liveness. Every request to enter the CS must be eventually satisfied
  - No deadlock, i.e. no endless waiting for a message that will never arrive
  - No starvation, i.e. no endless waiting for permission to enter the CS by one process, while other processes repeatedly execute the CS
- (3) Fairness. In the context of mutual exclusion the fairness property means that requests to enter the CS are granted in order of their occurrence (i.e. in happens-before order)

Approaches for the design of DME



- Permission-based algorithms
  - The right to enter a CS is formalized by obtaining “enough” permissions from other processes in the system
  - The safety property will be met if only one process out of all processes requesting the CS can get that many permissions
  - The liveness property is met by ordering requests by their logical timestamps or by using an acyclic precedence graph
- Token-based algorithms
  - The right to enter a CS is materialized by a token, which is unique in the whole system (safety is trivially satisfied)
  - The liveness property is met by moving the token “properly”: the perpetuum mobile and the token-asking methods
- Degenerate case – centralized algorithm

Centralized vs distributed algorithms



- Centralized algorithm
  - Most of the work is done by one process, while others play a minor role in achieving the overall result
  - Typically, this role boils down to providing the main process with the necessary data and receiving information from it
  - Asymmetric: different processes perform logically different operations
  - Client-server is the most suitable architecture
- Distributed algorithm
  - Each process plays the same role in achieving the overall result and in sharing the total load
  - Symmetric: all processes execute the same code and perform the same operations



Centralized algorithm

- One process is elected as a coordinator, which is solely responsible for granting permissions to enter the CS
  - E.g. the process with highest ID
- When process is going to enter CS, it sends REQUEST message to coordinator
- Incoming requests are queued by the coordinator according to the order in which they are received
- If no process executes CS (i.e. received request takes the first place in the queue), coordinator sends REPLY permission back
- After receiving REPLY, the process that requested access enters the CS

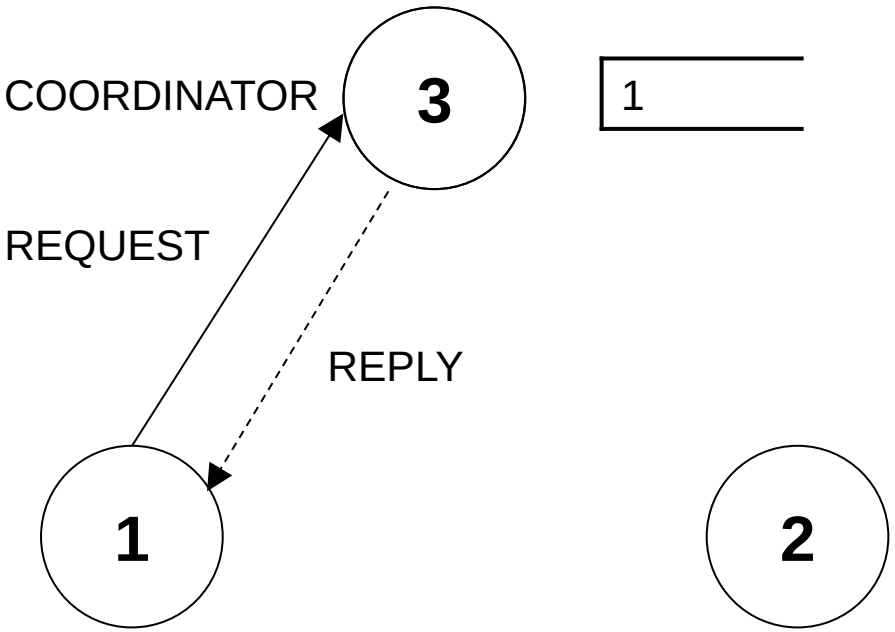


Centralized algorithm

- If access was granted to another process (the received request is not the first in the queue), coordinator either doesn't reply, or sends "access denied" message back to the requesting process
  - What is the difference?
- When a process exits CS, it sends RELEASE message to the coordinator, enabling other processes to enter the CS
- When receiving RELEASE messages, the coordinator removes the first item from the queue and sends REPLY message to the next process in the queue

Centralized algorithm

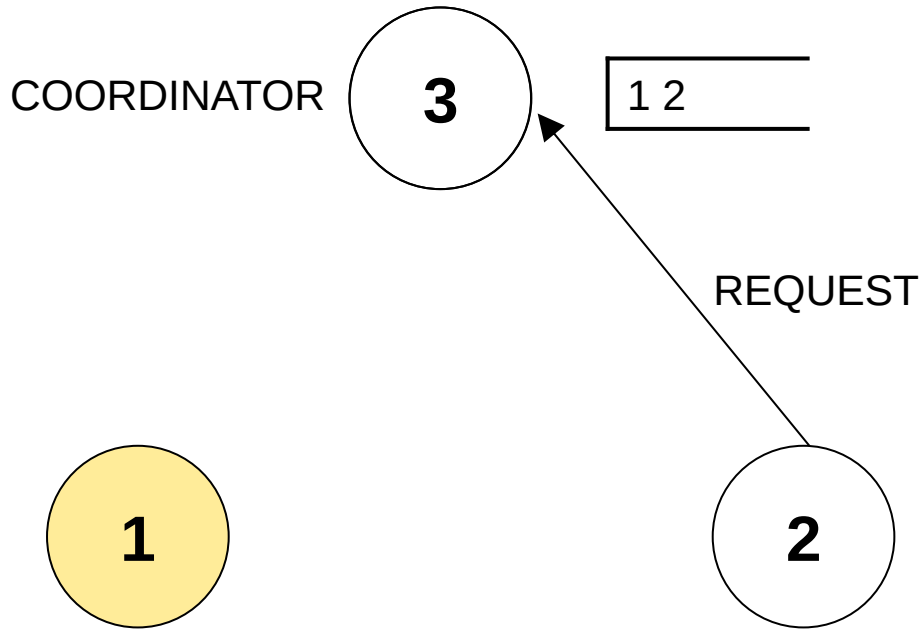
An example of a centralized algorithm execution



P1 sends a request to enter the CS and receives a reply

Centralized algorithm

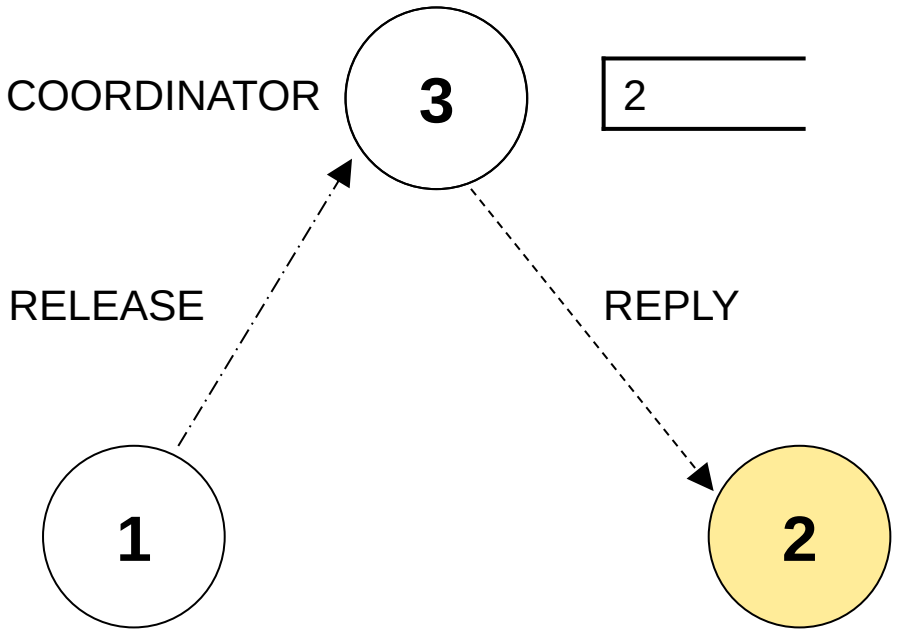
An example of a centralized algorithm execution



P1 enters the CS; P2 sends a request to enter the CS;
Coordinator puts the request from P2 in the queue;
P2 is blocked

Centralized algorithm

An example of a centralized algorithm execution



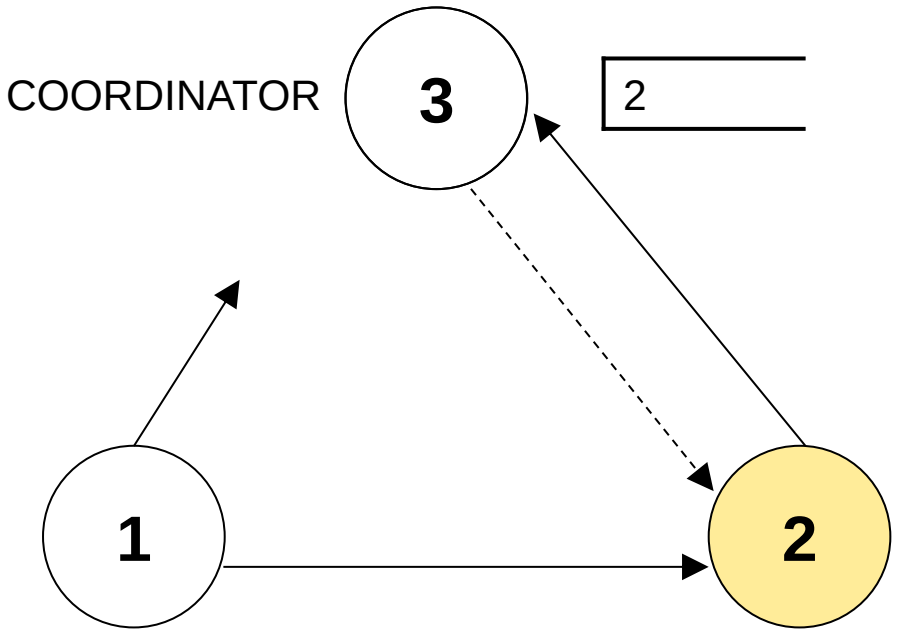
P1 exits the CS;
Coordinator sends a reply to P2;
P2 is got unblocked and enters the CS

Centralized algorithm

- Safety?
- Liveness?
- Fairness?

Centralized algorithm

Basically, not quite fair:



P1 sends a request to enter the CS and then sends a message to P2;
after receiving a message, P2 sends a request to enter the CS;
P2's request reaches the coordinator before P1's request

Lamport's distributed algorithm



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

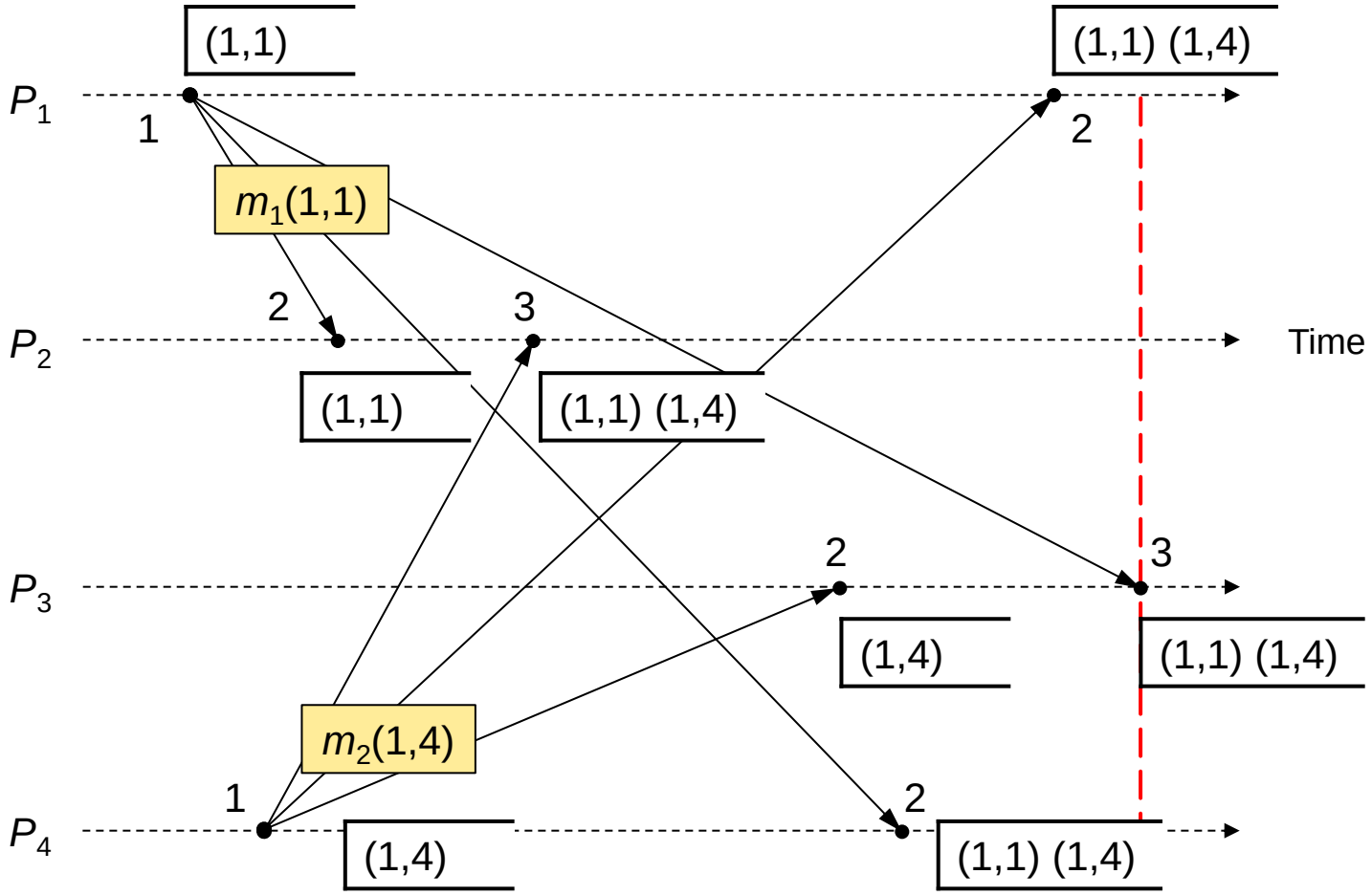
- Lamport's algorithm generalizes the centralized algorithm in the sense that requests to enter the CS are still placed in a queue and served from it in order
- The main difference is that the queue is not "owned" by the coordinator, i.e. it is not under the control of one single process, but can be considered as a separate object *shared* between all processes
- Another important difference is that requests are not served in an arbitrary order (order of receipt by the coordinator), but in the order of their logical timestamps
- Hence, if one request *happens before* another, then the first request will be satisfied before the second

Lamport's distributed algorithm



- To implement a view of a common shared queue, each process works with its local "copy" and uses logical timestamps to order the requests in its own "copy"
  - Thus, the request order is the same for all processes
- For each request to appear in each of these local "copies", a process broadcast the timestamped request and puts it to the local queue whenever it wants to enter the CS
- The process will get the right to enter the CS when the timestamp of its request turns out to be the lowest among all other requests
- But how can a process make sure that its request has the lowest timestamp?

Example of forming a distributed queue



For some time different requests will be at the head of local queues of different processes, which may lead to a violation of the mutual exclusion requirement

Lamport's distributed algorithm



- **Conclusion:** Before a process decides to enter the CS based on the state of its own queue, it should receive a message from every other process to ensure that there are no requests in the channels with timestamps less than the timestamp of its own request
- Here the algorithm requires that the channels must deliver messages in FIFO order, namely …
- … due to the FIFO channels and clock advancing protocol, such messages can be REPLY messages sent in response to a received request with a timestamp greater than the timestamp of the original request

Lamport's algorithm: Requesting the CS



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- Let Q_i be the local queue of process P_i which stores all requests to enter the CS in order of their timestamps
- When P_i wants to enter the CS, it broadcasts REQUEST(L_i, i) and puts the request in its own local queue Q_i
- When P_j receives REQUEST(L_i, i) from P_i, it puts the request in its own queue Q_j and sends REPLY(L_j, j) back to process P_i with the value of its logical time (L_j, j)

Lamport's algorithm: Entering the CS



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- Process P_i enters the CS when the following two conditions are met:
- (1) REQUEST(L_i, i) of process P_i has the *smallest* timestamp value among all requests in its own queue Q_i
- (2) P_i has received a message with timestamp larger than (L_i, i) from all other processes
  - Meeting this requirement ensures that P_i is aware of all requests that precede its current request

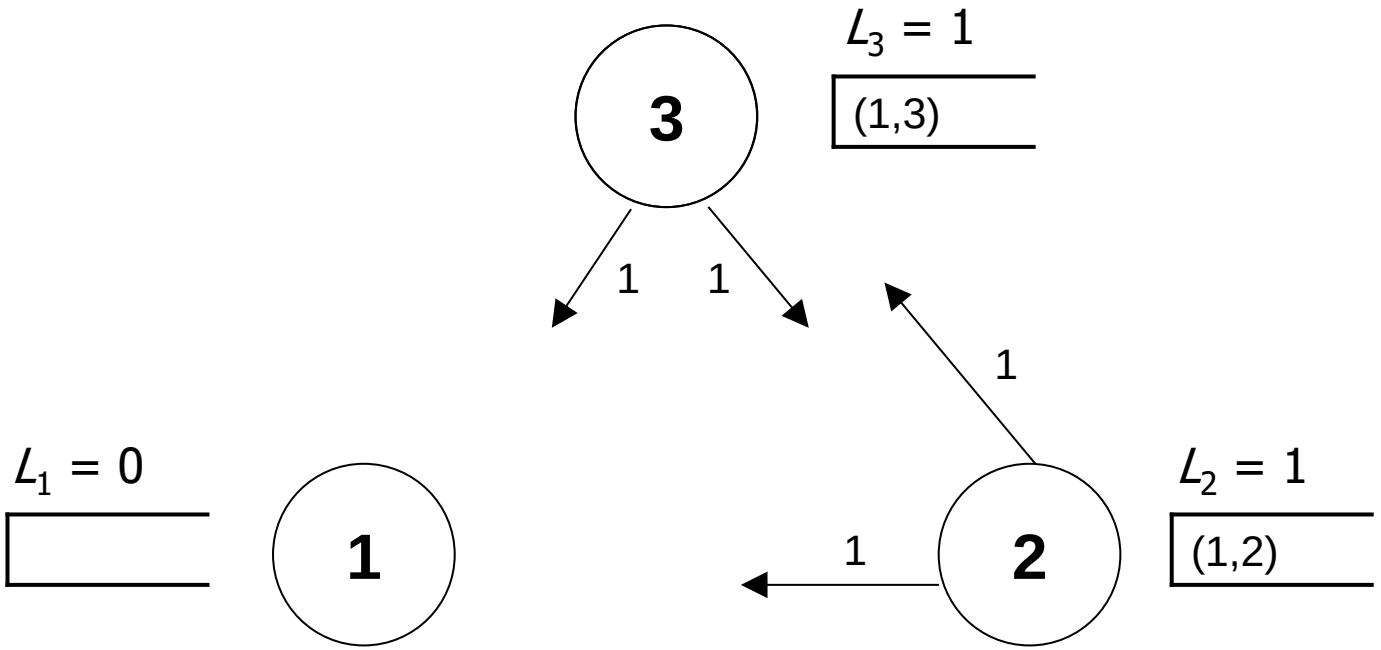
Lamport's algorithm: Releasing the CS



- Process P_i removes its request from its own queue Q_i and broadcasts RELEASE(L_i, i) with its logical timestamp
- Upon receiving RELEASE(L_i, i) process P_j removes request of P_i from its queue Q_j
- After process P_j removes request of P_i from Q_j, the timestamp of its own request may become the smallest in Q_j, and P_j can count on entering the CS

Lamport's algorithm

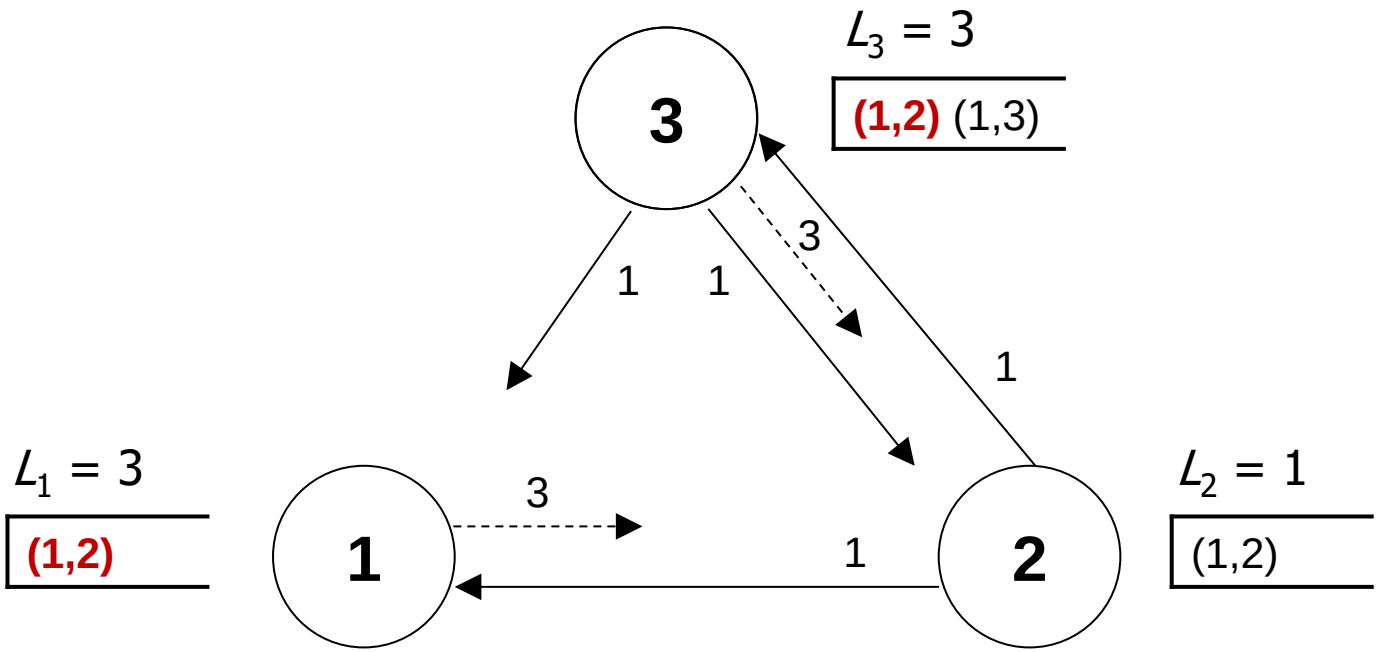
An example of Lamport's algorithm execution



P2 and P3 send requests to enter the CS

Lamport's algorithm

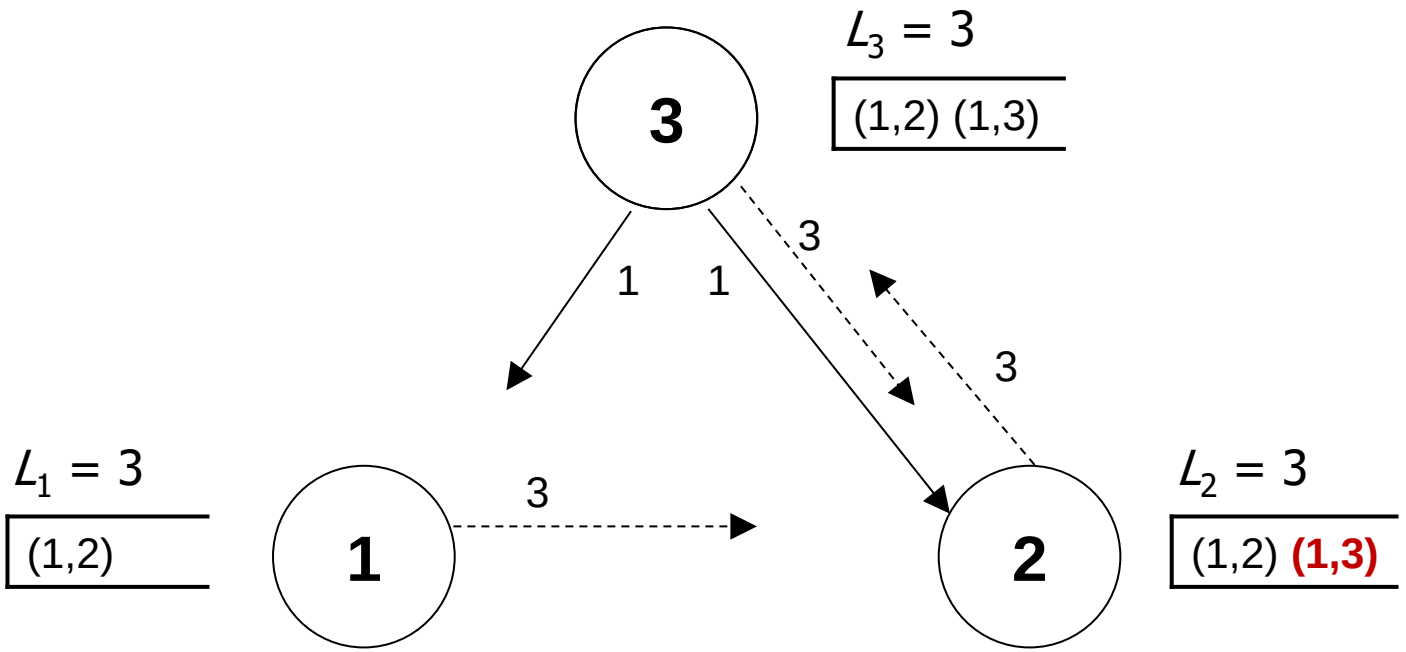
An example of Lamport's algorithm execution



P1 and P3 send replies

Lamport's algorithm

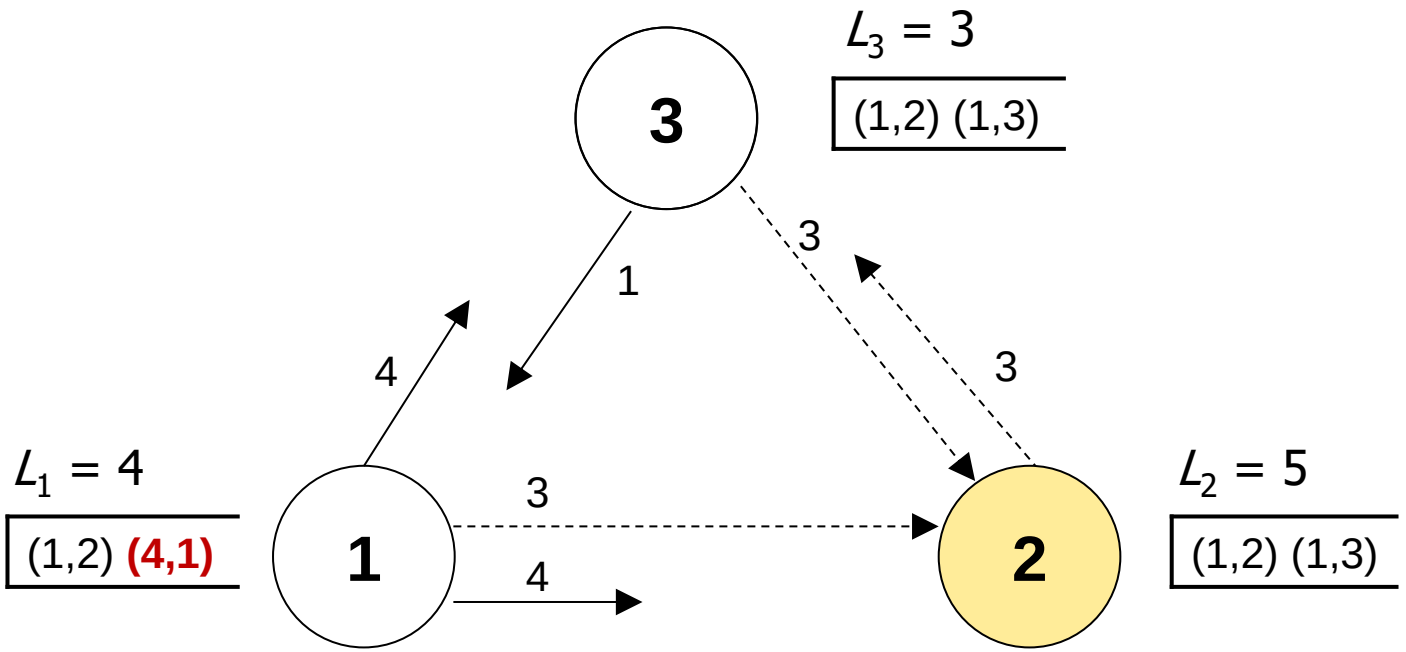
An example of Lamport's algorithm execution



P2 sends reply

Lamport's algorithm

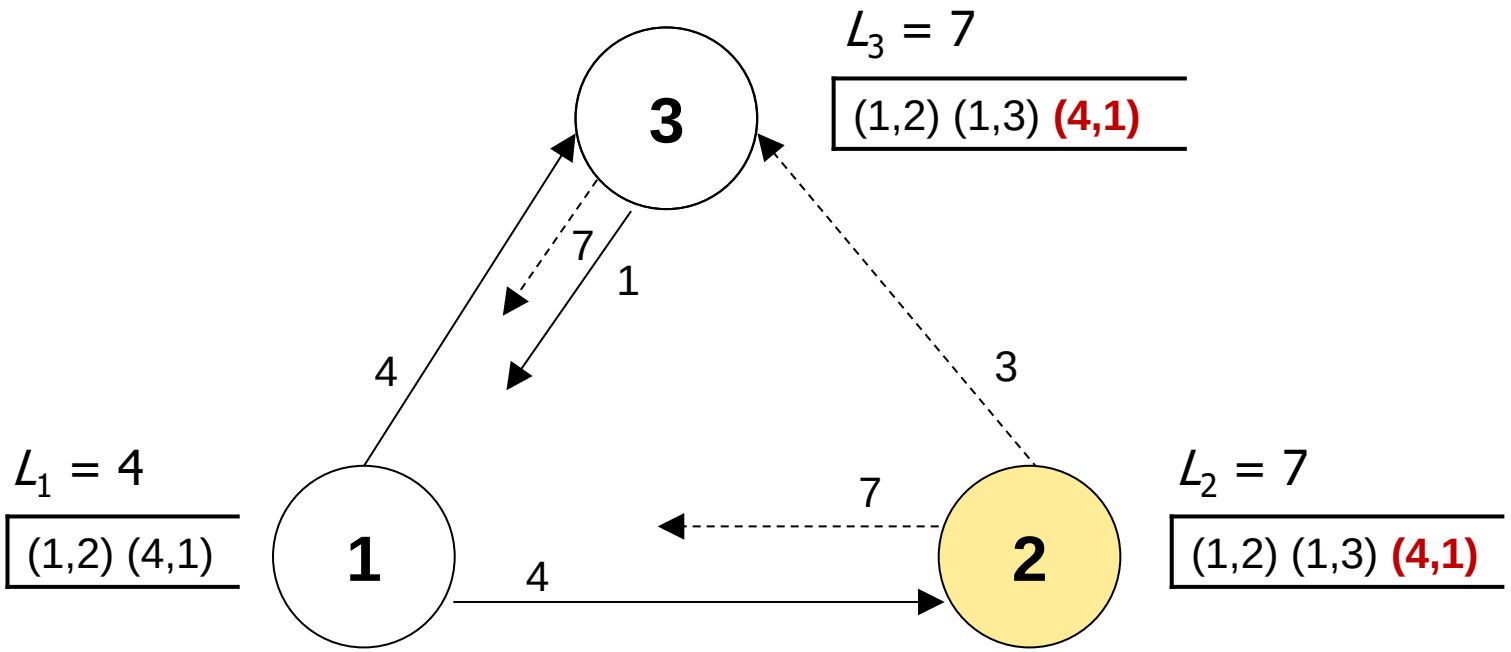
An example of Lamport's algorithm execution



P2 enters the CS;
P1 sends requests to enter the CS

Lamport's algorithm

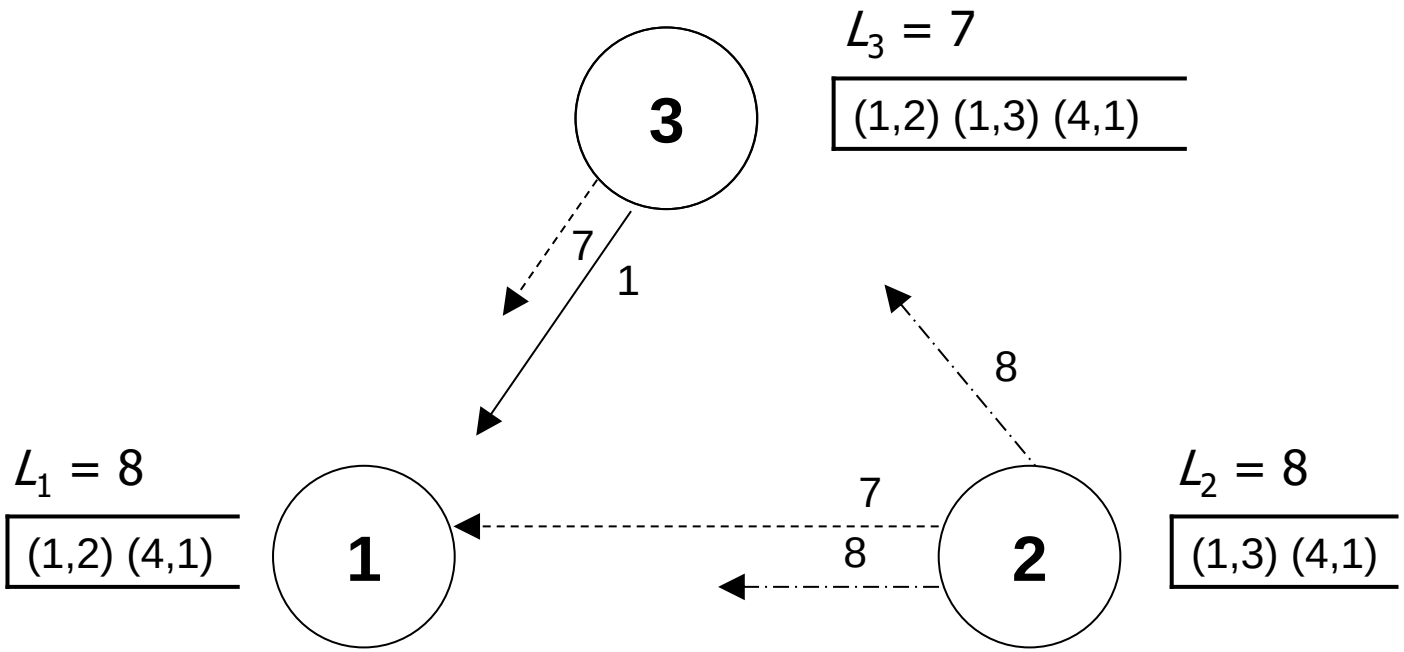
An example of Lamport's algorithm execution



P2 and P3 send replies

Lamport's algorithm

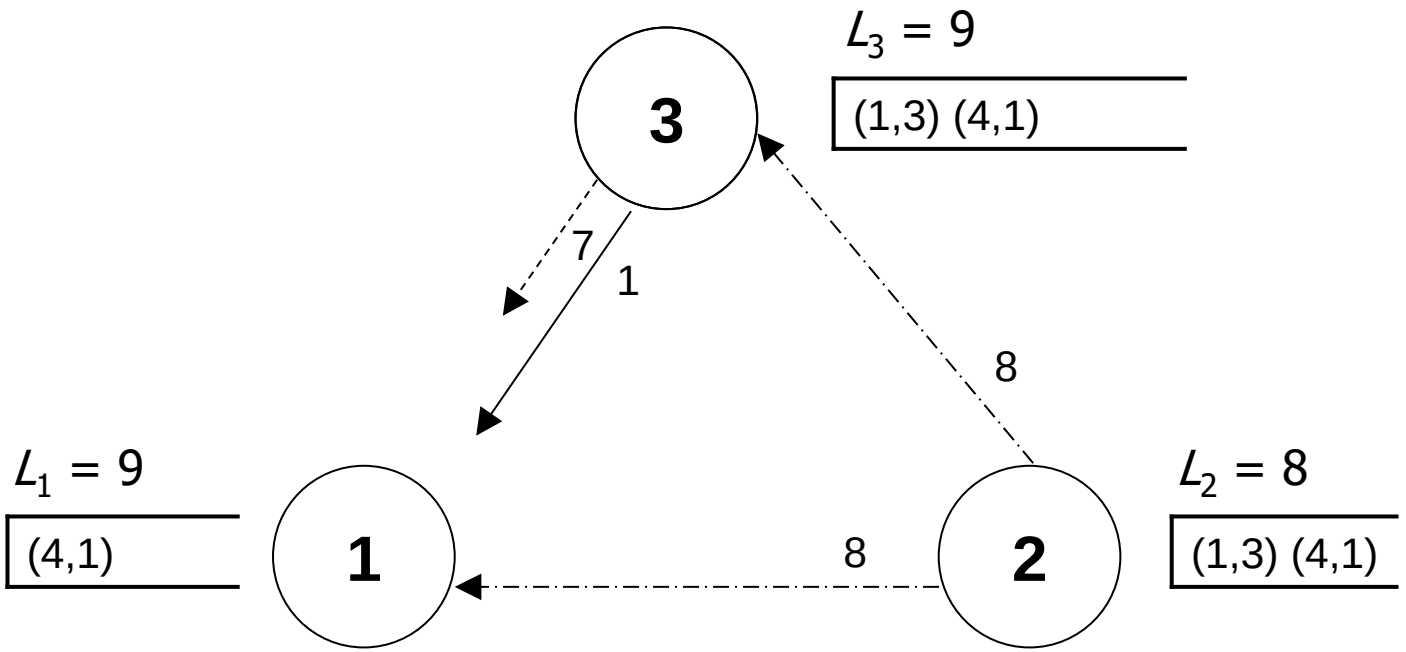
An example of Lamport's algorithm execution



Due to the FIFO property of the channel
the reply from P3 can't be received by P1 before its request;
P2 exits the CS

Lamport's algorithm

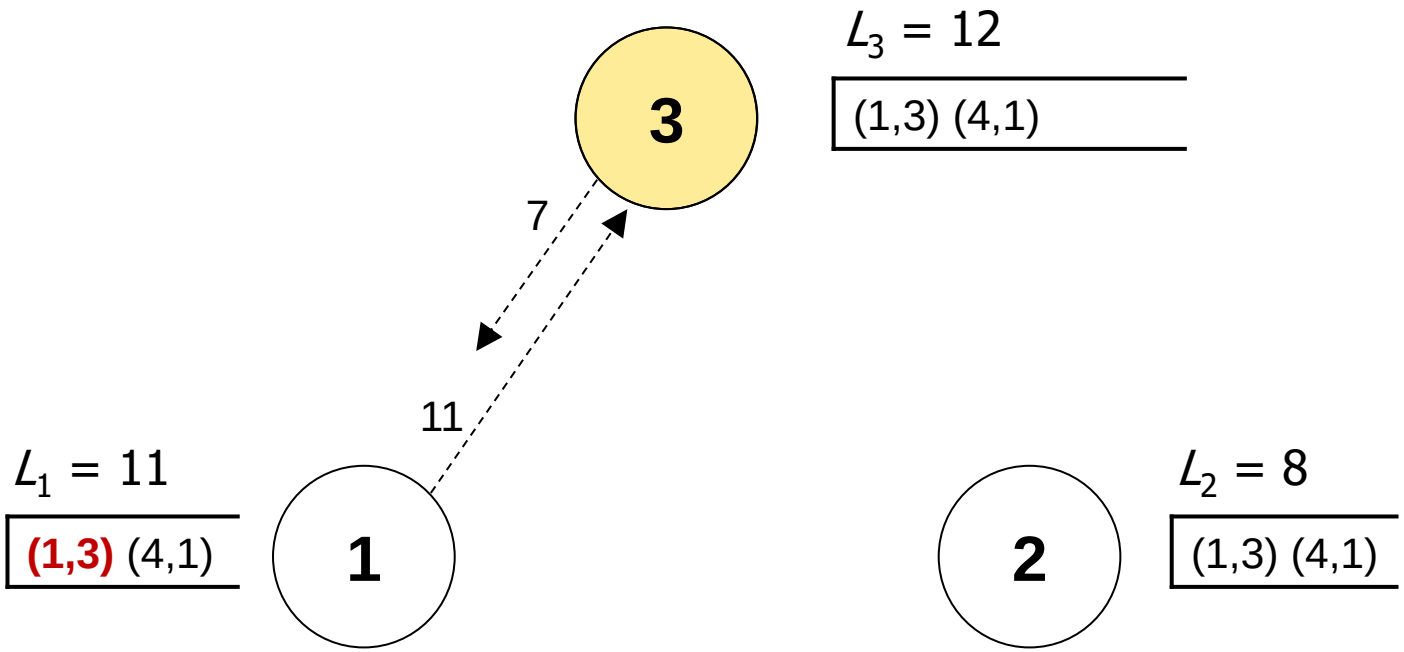
An example of Lamport's algorithm execution



Due to the FIFO property of the channel
the reply from P3 can't be received by P1 before its request;
P2 request is removed from the P1 and P3 queues

Lamport's algorithm

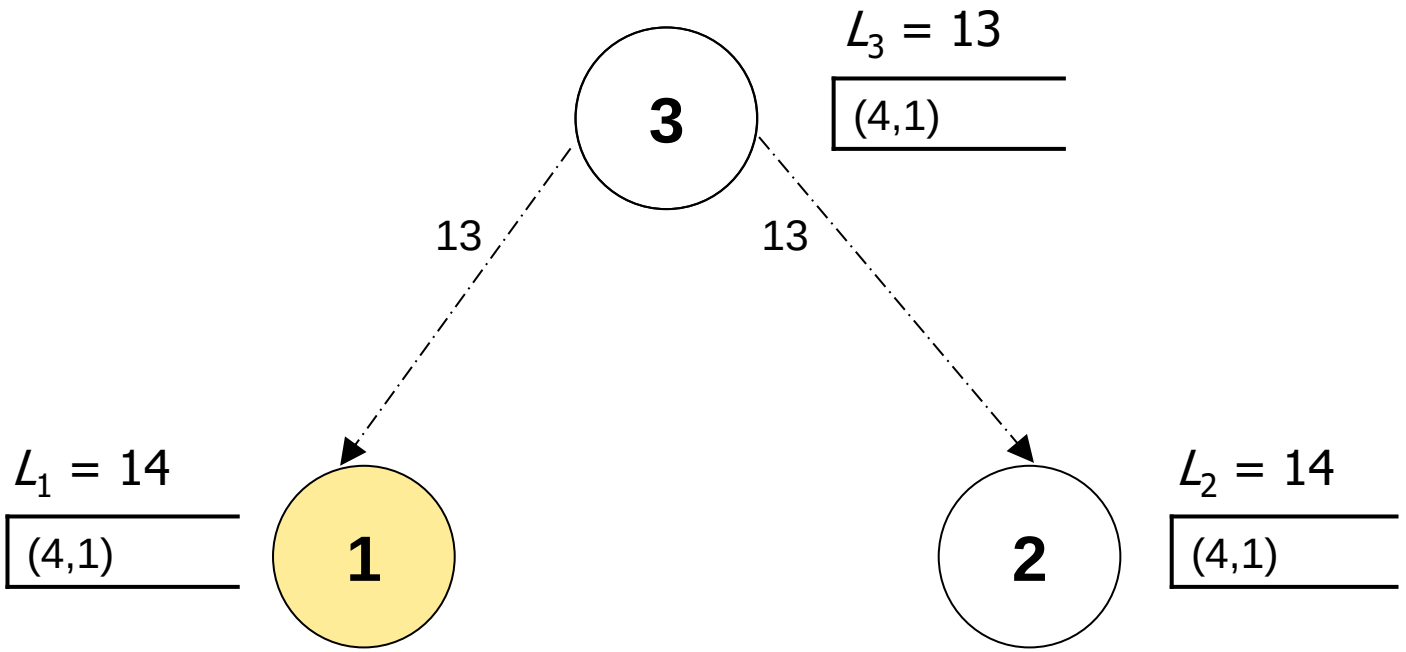
An example of Lamport's algorithm execution



P3 request reaches P1 and takes the first place in P1's queue;
P1 sends a reply;
on getting the reply from P1, P3 enters the CS

Lamport's algorithm

An example of Lamport's algorithm execution



P3 exits the CS;
P3's request is removed from all queues;
P1 enters the CS

Lamport's algorithm properties



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- Safety? By contradiction… Suppose P_i and P_j run in the CS concurrently, which means that:
  - (1) the timestamp of the P_i request turned out to be the smallest in its queue Q_i, and the timestamp of the P_j request was the smallest in the queue Q_j
  - (2) both processes received reply messages from each other with a timestamps bigger than the timestamps of their requests
- Let assume that P_i's request has smaller timestamp than the request of P_j
  - Relying on the channels FIFO property and clock advancing protocol, it can be argued that at the time P_j received a reply from P_i, the request from P_i should have already been in its queue Q_j, and the request P_j could not have the smallest timestamp in Q_j

Lamport's algorithm properties



HDU-ITMO Joint Institute
杭州电子科技大学 圣光机联合学院

- Liveness?
  - There are no more than (N – 1) requests from other processes with lower timestamps before P_i's request in the queue
  - When leaving the CS, P_j sends out a RELEASE message, which removes P_j's request from all copies of the queue (including Q_i)
  - Subsequent requests to enter the CS from P_j will have timestamps bigger than the timestamp of P_i's request
  - Therefore, in a limited number of steps, the timestamp of P_i's request will become the smallest one in Q_i and P_i will eventually be able to enter the CS
- Fairness?
  - By definition …

Lamport's algorithm properties



- The algorithm is distributed
  - Symmetric: all processes follow the same rules and each process independently follows these rules and makes a decision to enter the CS only based on its local information
  - There is no central synchronizing process or central storage
  - The approach can be generalized to build a RSM, i.e. a fault-tolerant distributed server (assuming failure detection is perfect)
- The algorithm requires $3(N-1)$ messages per CS invocation
- Do all messages / events need to have a logical clock timestamp?
- Are all of the above messages always required?

An optimization of Lamport's algorithm



- In certain situations REPLY messages can be omitted
- If process P_j receives REQUEST(L_i, i) from P_i, after it has send its own REQUEST(L_j, j) with timestamp (L_j, j) > (L_i, i), then process P_j need not send a REPLY message to P_i
- This is because such a REQUEST from P_j performs the function of a REPLY message
  - Such a request will assure P_i that it will not receive any more requests from P_j with a timestamp less than (L_i, i)
- With this optimization Lamport's algorithm requires between 3(N−1) and 2(N−1) messages per CS invocation

Ricart–Agrawala algorithm



- R-A algorithm is an attempt to optimize Lamport's algorithm by combining the functions of RELEASE and REPLY messages in one message
  - Before a process decides to enter the CS it still needs to make sure that there are no requests in the channels that precede its current request
  - Such process may rely not on the timestamps of the received messages, but instead on the number of REPLIES received, so the timestamps in REPLIES can be omitted
- Channels don't need to have FIFO property
- The process needs to collect permissions from *all* other processes in order to enter the CS

Ricart–Agrawala algorithm



- A process wishing to enter the CS broadcasts its REQUEST to all other processes and waits for permissions in the form of REPLY messages
- If other processes execute outside the CS they give their permission by sending REPLY immediately
- If a process executes the CS, it postpones its REPLY until it leaves the CS so that not to break the safety property
- If other processes are also requesting the CS then as in Lamport's algorithm we use logical timestamps to resolve the conflict
  - Request with lowest timestamp will get the right to enter the CS first

Ricart–Agrawala algorithm



- If a process P_i requesting the CS receives another request from another process P_j, it needs to compare the timestamp (L_i, i) of its own request with the timestamp (L_j, j) of the incoming request, and
  - if $(L_i, i) < (L_j, j)$ then P_i postpones sending REPLY (P_i wins)
  - if $(L_i, i) > (L_j, j)$ then P_i sends REPLY to P_j right away (P_i loses)
- Each process P_i maintains DR_i [1..N] array which contains flags of deferred replies, initialized to zero
- If P_i defers sending REPLY message to P_j, it sets DR_i [j] = 1, and after it has sent REPLY to P_j, it sets DR_i [j] = 0

R-A algorithm: Requesting the CS



- When P_i wants to enter the CS, it broadcasts REQUEST(L_i, i)
- When P_j receives REQUEST(L_i, i) from P_i, P_j sends a REPLY message to P_i if it is neither requesting nor executing the CS, or if it is requesting the CS and the timestamp of its own request is larger than the timestamp of P_i's request
- Otherwise, the reply is deferred and P_j sets $DR_j[i] = 1$

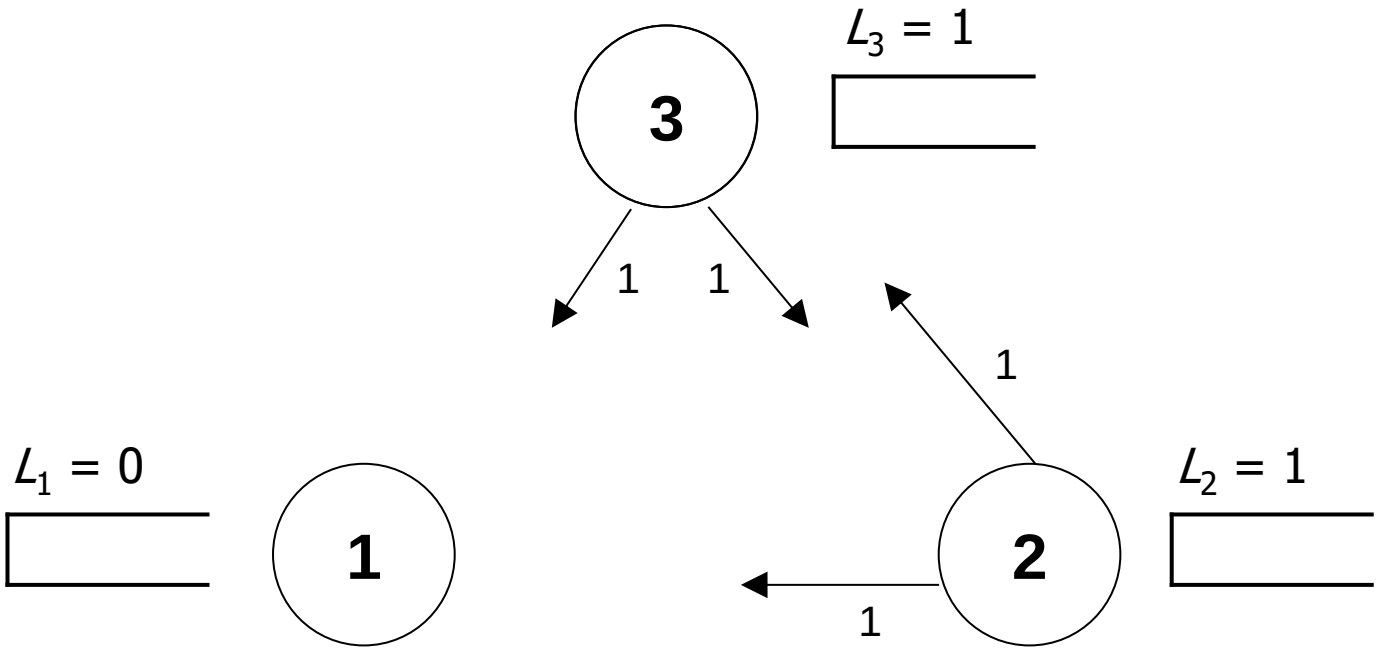
R-A: Entering and releasing the CS



- Entering the CS: Process P_i enters the CS when it has received REPLY messages from all other processes
- Exiting CS: When process P_i exits the CS it sends all deferred REPLY messages to waiting processes, i.e. to all processes P_j, for which $DR_i[j] = 1$, and then sets $DR_i[j] = 0$

Ricart–Agrawala algorithm

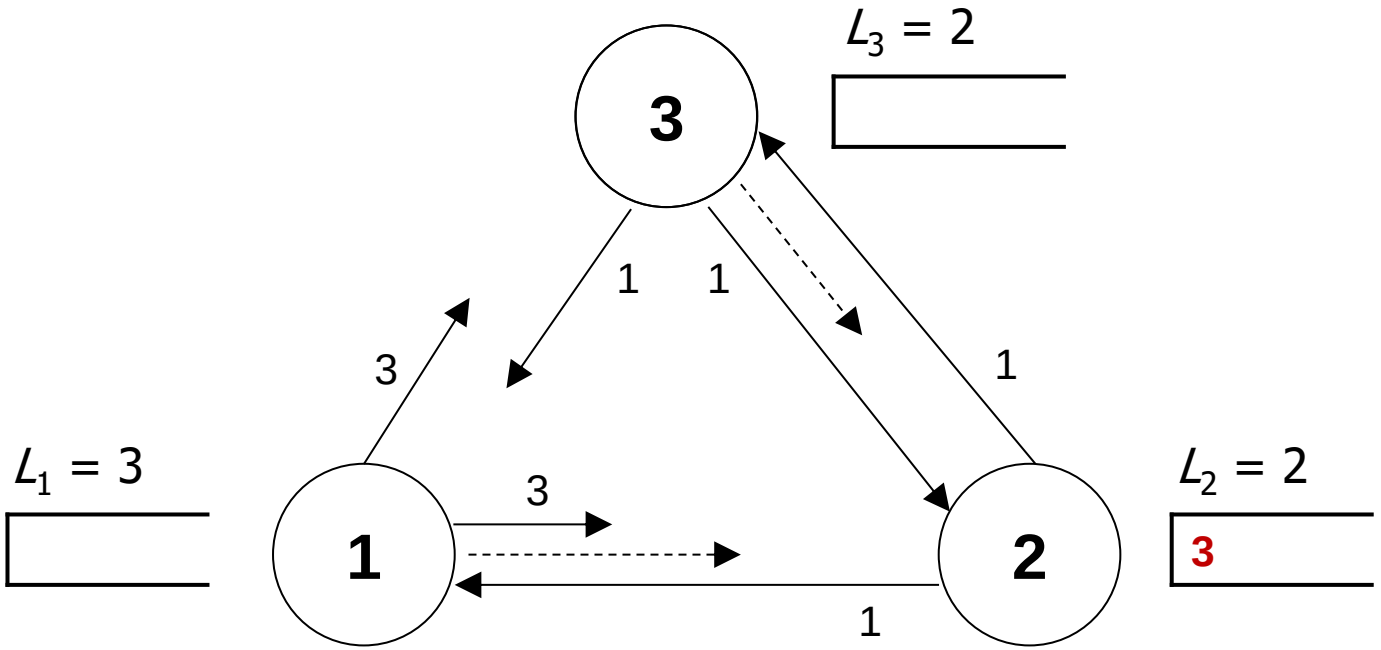
An example of Ricart-Agrawala algorithm execution



P2 and P3 send requests to enter the CS

Ricart–Agrawala algorithm

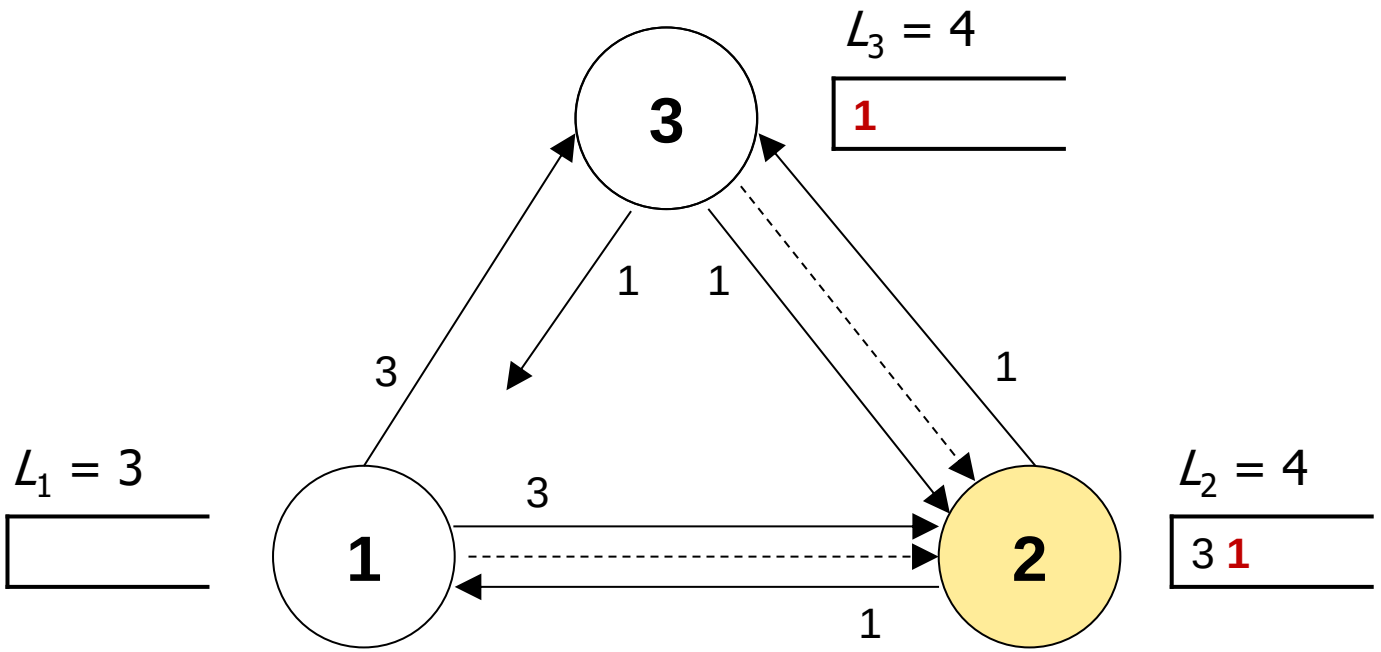
An example of Ricart-Agrawala algorithm execution



P1 and P3 send replies to P2; P2 defers its reply to P3
P1 sends requests to enter the CS

Ricart–Agrawala algorithm

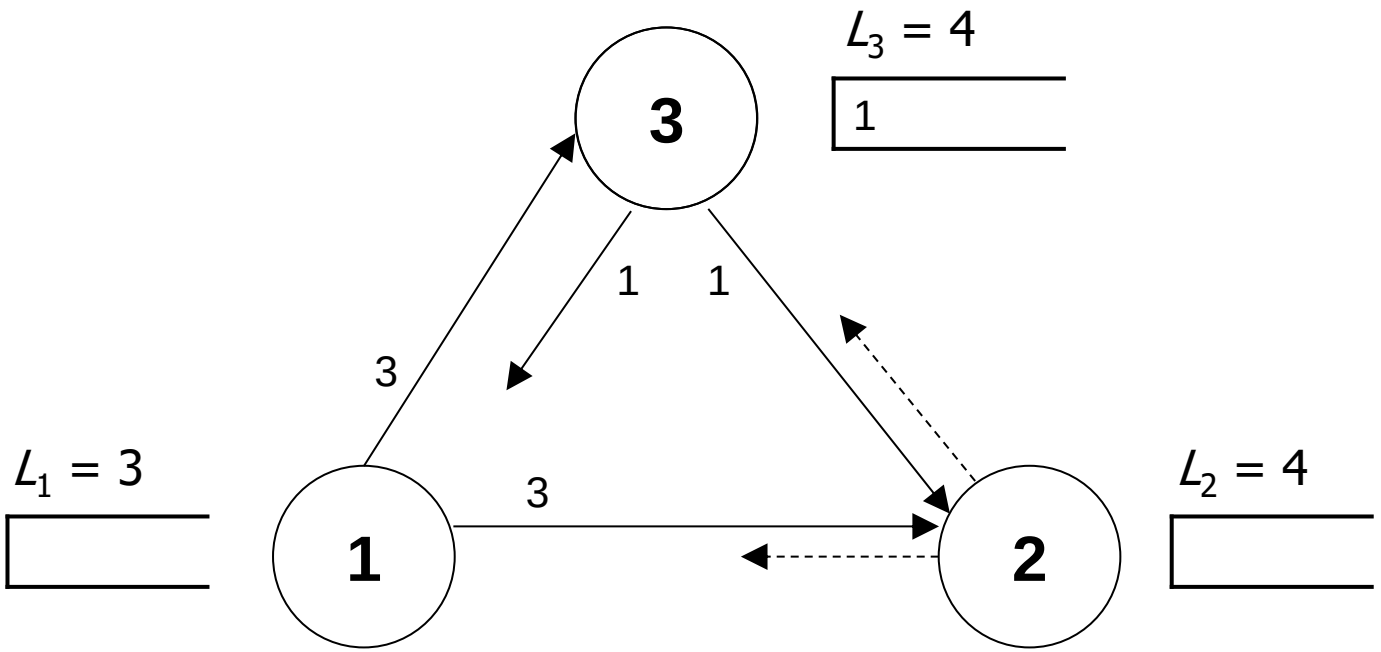
An example of Ricart-Agrawala algorithm execution



P2 gets replies from all other processes and enters the CS
P1's requests reach P2 and P3
P2 and P3 defer their replies

Ricart–Agrawala algorithm

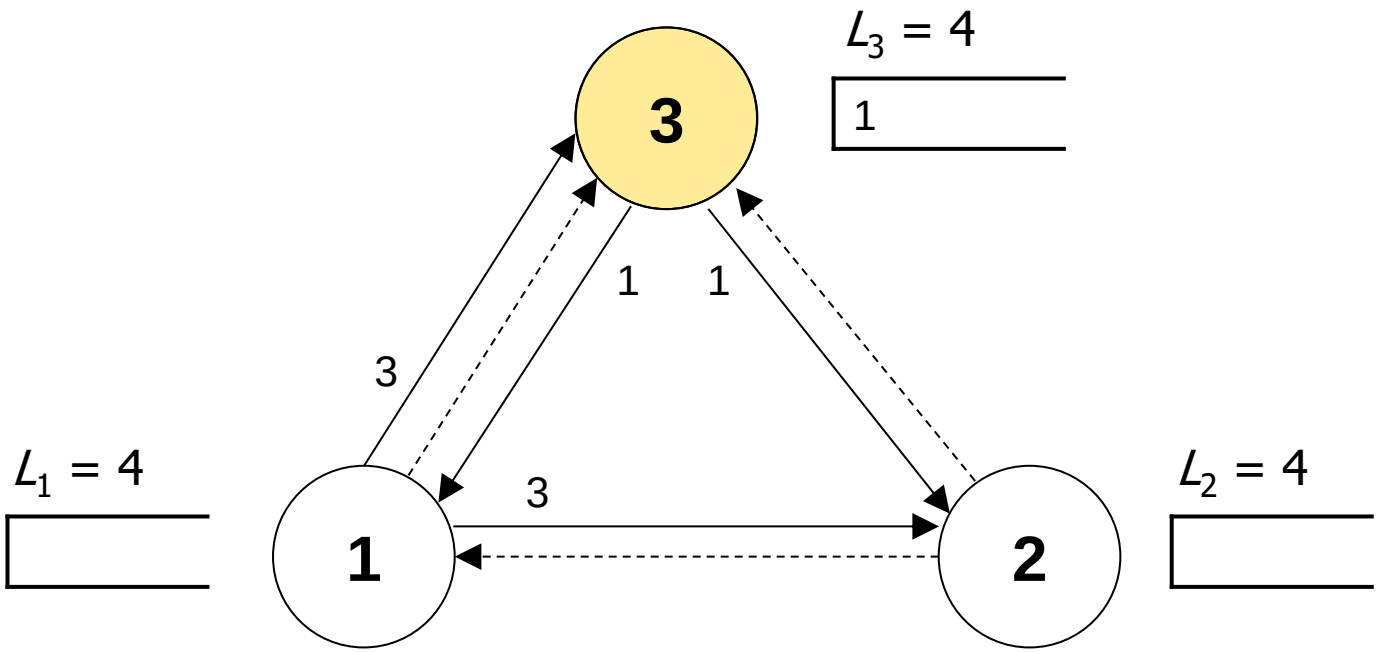
An example of Ricart-Agrawala algorithm execution



P2 exits the CS
P2 sends out deferred replies to P3 and P1

Ricart–Agrawala algorithm

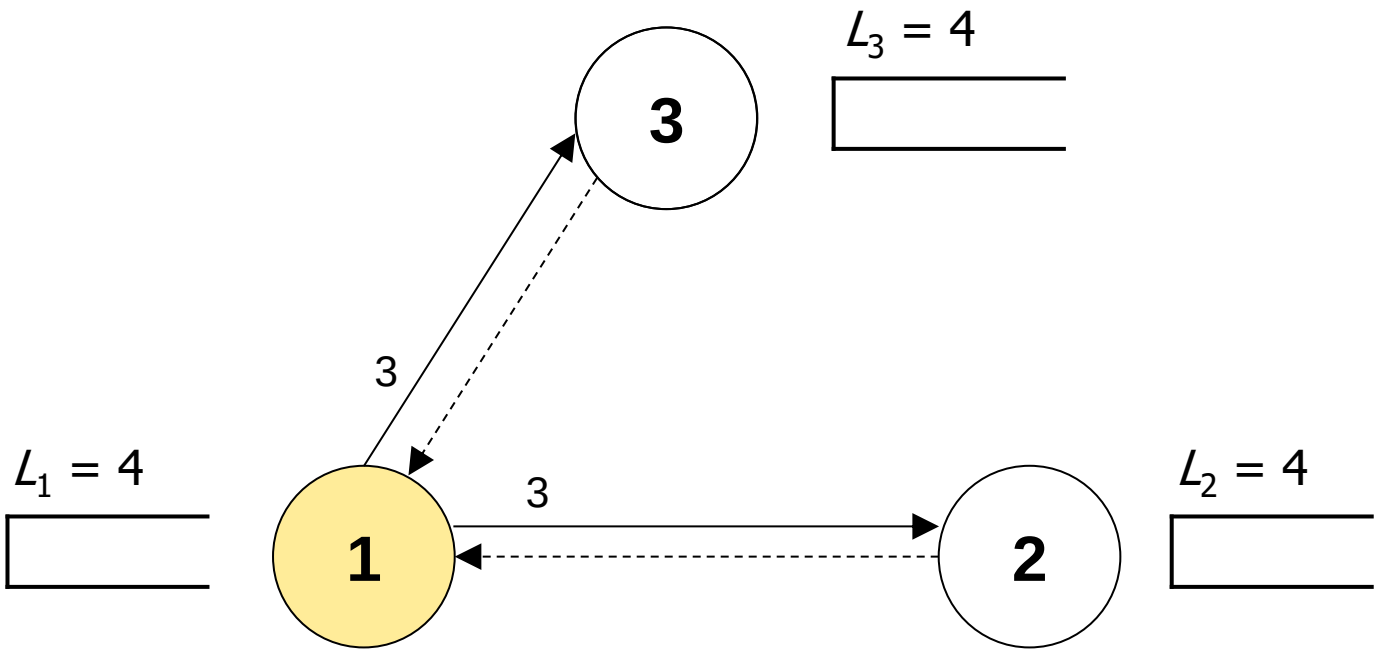
An example of Ricart-Agrawala algorithm execution



P2's deferred replies reach P1 and P3
P3's request reaches P1, P1 sends reply to P3
P3 gets replies from P2 and P1 and enters the CS

Ricart–Agrawala algorithm

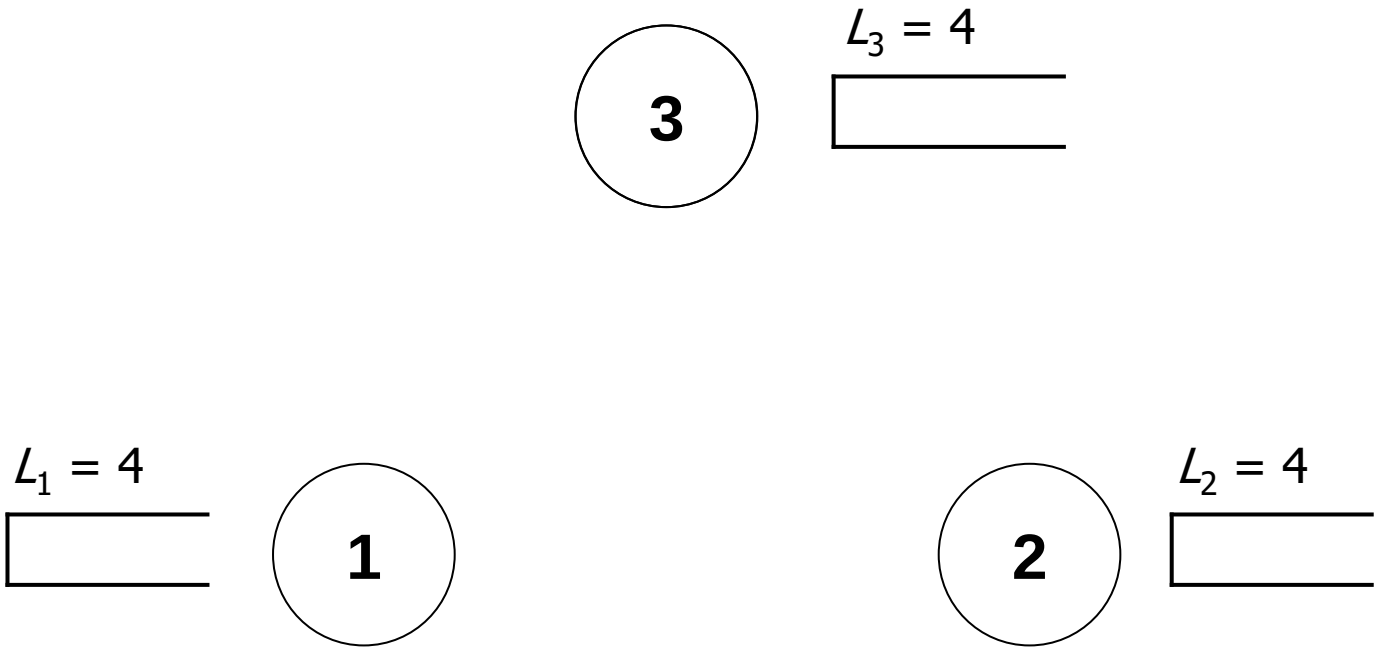
An example of Ricart-Agrawala algorithm execution



P3 exits the CS
P3 sends deferred reply to P1
P1 gets all replies and enters the CS

Ricart–Agrawala algorithm

An example of Ricart-Agrawala algorithm execution



P1 exits the CS
P1 has no deferred replies, so P1 has nothing to do

Ricart–Agrawala algorithm properties



- Safety? By contradiction… Suppose P_i and P_j run in the CS concurrently, and let assume that P_i's request has smaller timestamp than the request of P_j
  - P_j can enter the CS only when it receives REPLY messages from all other processes
  - P_j's request reached P_i after P_i sent its own request; otherwise, timestamp of the P_i's request would be greater than timestamp of the P_j's request according to the clock advancing protocol
  - But P_i can't send REPLY to P_j when requesting the CS and having request with smaller timestamp, until P_i enters and exits the CS
  - Thus, we get a contradiction with the assumption that P_j received REPLY messages from all processes, including P_i

Ricart–Agrawala algorithm properties



- Liveness?
  - Lets build a chain of processes, waiting for each other: P_i waits for P_j, P_j waits for P_k, P_k waits for P_m, etc.
  - This chain is finite and is acyclic: the processes are arranged in it in descending order of timestamps of their requests
  - The last process P_l in this chain with the smallest timestamp of its request either executes the CS, or will get permissions from all other processes and will be able to enter the CS
  - When leaving the CS, P_l will send REPLIES to all waiting processes, including the process immediately to the left in the chain, thereby allowing this process to enter the CS
  - Therefore, in a limited number of steps each process will be the last in the waiting chain and will be able to enter the CS

Ricart–Agrawala algorithm properties



- Fairness?
  - As in Lamport's algorithm, access to the CS is granted strictly in accordance with the timestamps of the requests, i.e. in the order consistent with the partial causal order
  - This implies that after the REQUEST message from P_i is received by all processes, none of them will be able to enter the CS more than once ahead P_i
  - In Lamport's algorithm requests to the CS were *explicitly* queued, while in Ricart-Agrawala algorithm processes requesting the CS are *implicitly* ordered into an acyclic chain, in which each process waits for permissions from the processes to the right of it ...
- The algorithm requires $2(N-1)$ messages per CS invocation